
Secure AES-128 Coprocessor with TPM-Style Logic Locking

Carly Rosenstrauch, William Neel
University of Massachusetts Amherst

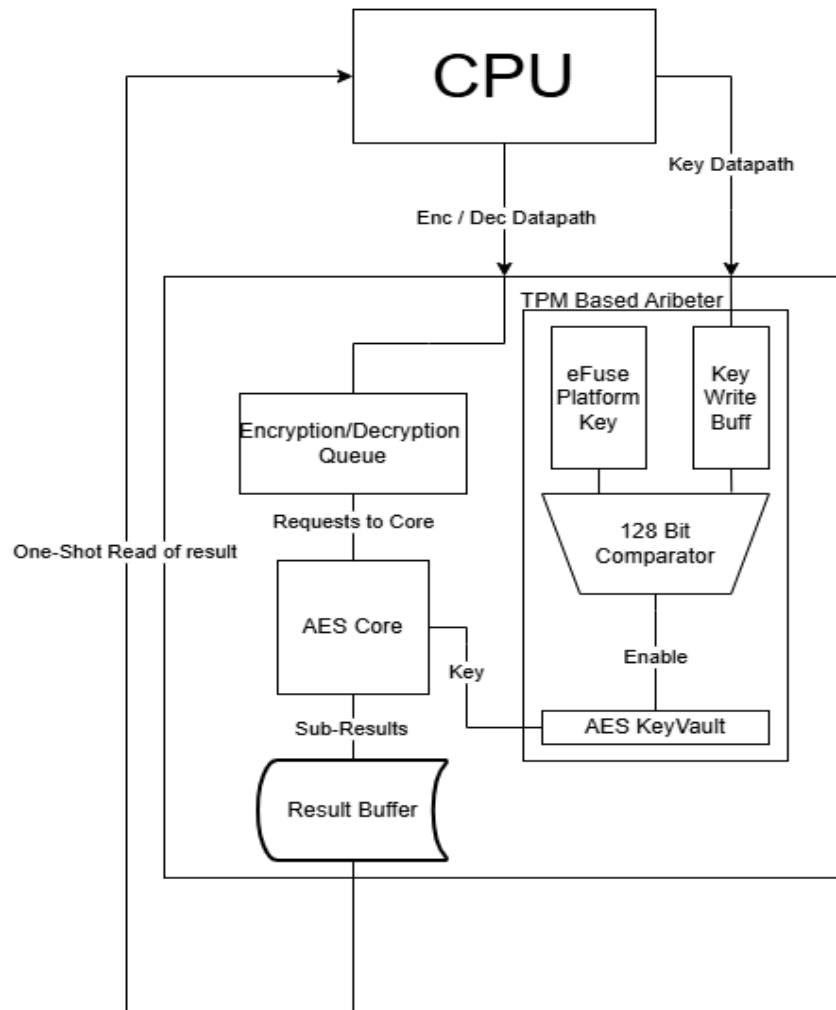
1.) Detailed Project Plan

This project aims to build and test a secure AES-128 coprocessor protected by a TPM-style command interface and FSM-based logic locking. While the AES encryption engine itself uses a standard iterative design, the unique part of this project is the security wrapper built around it. This wrapper manages authentication, secures the keys, and strictly gates execution. Essentially, the coprocessor acts as a hardware Root of Trust and it will remain completely non-operational until the correct 128-bit activation sequence unlocks it.

Development is split into two main pieces: the AES datapath and the TPM controller. To balance area efficiency with predictable latency, the AES engine will use an iterative structure to cycle through the standard transformations (SubBytes, ShiftRows, MixColumns, AddRoundKey, and key expansion). We will verify this module against standard AES test vectors before moving forward. At the same time, we will build the TPM controller to act as the middleman between the host and the AES engine, handling commands like STATUS, UNLOCK, LOAD_KEY, COMMIT_KEY, ENCRYPT, and READ_RESULT. System activation will be managed by a multi-stage FSM moving through LOCKED, UNLOCK_SHIFT, UNLOCKED, and TRAPPED states. If an incorrect unlock sequence is attempted, the system falls into a persistent TRAPPED state that requires a complete zeroization to reset. Most importantly, the encryption datapath is strictly hardware-gated to ensure no unauthorized cryptographic operations can occur.

For integration, the main focus is making sure the handshake between the TPM and AES engine is rock-solid and that the security policies cannot be bypassed. We will verify the design in steps: first unit testing the AES logic and the FSM behavior, then validating the key vault and zeroization logic at the subsystem level. Finally, we will run full-system simulations to prove the unlock sequence works and the trap states trigger correctly. We will specifically test what happens if an encrypt command is sent while the system is locked or trapped to confirm the gating logic holds up.

Finally, we need to evaluate how much hardware overhead this security wrapper actually adds. To do this, we will synthesize both a baseline standalone AES core and our secured version using the exact same constraints. By comparing the post-synthesis reports, we can measure the impact on clock frequency, critical path delay, and total area utilization. We are targeting an operating frequency between 10 and 20 MHz, with an area overhead of no more than roughly 25% compared to the unprotected AES core. This comparison will give us a clear, quantitative look at the true cost of adding architectural security enforcement.



2.) Literature Review

Cryptographic hardware has historically prioritized performance metrics such as throughput, latency, and silicon area efficiency, often placing less emphasis on structural security and intellectual property protection. However, the increasing prevalence of hardware cloning, IP theft, and invasive reverse engineering has driven significant research into architectural countermeasures that embed security directly into digital design [4], [5].

The Advanced Encryption Standard (AES), defined by the National Institute of Standards and Technology in FIPS-197, establishes the functional and mathematical foundation for secure symmetric encryption [1]. The standard specifies the algorithmic operations, including substitution, permutation, and key expansion processes. While cryptographically robust, the specification does not address hardware level protections against physical inspection, netlist extraction, or unauthorized functional activation [1]. As a result, a correctly implemented AES core may remain vulnerable to structural exploitation despite algorithmic security.

Trusted computing architectures address this limitation. The Trusted Computing Group TPM 2.0 specification defines hardware roots of trust, secure key storage, and authenticated command processing [2], [3]. TPM modules isolate cryptographic operations behind controlled interfaces, preventing direct datapath access [2], [3].

Logic locking and hardware obfuscation techniques extend these protections into the design fabric itself. Research such as *Security Analysis of Logic Obfuscation* (IEEE) demonstrates that finite state machine (FSM)–based locking sequences can render extracted netlists functionally

inoperable without correct activation keys [4]. These approaches introduce additional state elements, key comparators, and enable gating structures that bind circuit functionality to secret activation conditions [4].

Further work, including *Hardware IP Protection against Confidentiality Attacks and the Evolving Role of CAD Tools* (ACM/IEEE), highlights emerging threats at the design automation level [5]. Identified risks include scan-chain leakage, structural inference attacks, SAT-solver-based deobfuscation, and side-channel-assisted reverse engineering [5]. These studies emphasize that no single protection mechanism is sufficient in isolation.

This project synthesizes these domains by embedding TPM-like command authorization directly into an AES coprocessor, creating a unified hardware root of trust.

3. Motivation Behind the Project

Cryptographic accelerators such as AES-128 cores are widely deployed in modern digital systems, both as standalone IP blocks and as integrated components within larger system-on-chip architectures. While these implementations typically emphasize functional correctness and performance, they often provide limited protection against hardware reverse engineering, IP theft, or unauthorized activation. In ASIC and FPGA development environments, extracted netlists and cloned hardware designs present realistic security risks.

The motivation for this project is to integrate architectural security mechanisms directly into the cryptographic coprocessor itself. Rather than modifying the AES algorithm, we introduce security through structural isolation and controlled activation using a Trusted Platform Module (TPM)-style command processor and a multi-stage finite state machine (FSM)-based logic locking mechanism. The system is designed to function as a hardware Root of Trust: even if the encryption datapath is physically inspected or reverse-engineered, it remains nonfunctional without successful authentication through a 128-bit activation sequence. This approach emphasizes hardware-level security enforcement while maintaining compatibility with standard AES functionality.

4.) Tools and Platforms

Design & RTL Development

Languages:

- Verilog and SystemVerilog are being used for register-transfer level (RTL) design and system modeling.

Design Methodology:

- Modular, synthesizable RTL
- Hierarchical subsystem decomposition
- Separation of datapath and control-path logic

This approach ensures compatibility with downstream synthesis and physical design tools while maintaining design clarity and verification accessibility.

Functional validation of all RTL modules is being performed using:

Synopsys VCS - Industry-standard simulation and verification environment.

Assertion-Based Verification

In addition to traditional testbench stimulus, SystemVerilog Assertions (SVAs) are being used to enforce key protocol and security invariants within the design. Assertion-based verification provides a mechanism for continuously monitoring critical conditions during simulation and immediately flagging violations.

The SVAs focus primarily on validating the control-path behavior surrounding the TPM-style command interface and logic-locking FSM. Several classes of properties are monitored:

Access Control Assertions

- Encryption operations must not begin while the system is in the **LOCKED** or **TRAPPED** states.
- The AES datapath enable signal must only be asserted when the FSM is in the **UNLOCKED** state.

Command Sequencing Assertions

- Command ordering constraints ensure that **LOAD_KEY** must occur before **COMMIT_KEY**.
- Encryption requests are only valid after successful system activation.

Key Vault Safety

- Key readout is only permitted when the vault output-enable condition is satisfied.
- Key registers must clear immediately following a zeroization event.

FSM Safety Properties

- Illegal state transitions are prevented.
- Incorrect unlock attempts trigger the **TRAPPED** state as expected.

These assertions allow the verification environment to automatically detect violations of the intended security policies and interface protocols during simulation. Assertion failures immediately halt simulation and provide diagnostic information for debugging.

Compute Platform: UMass ECE VLSI CAD Servers, Linux simulation environment

5.) Timeline

- Week 1: Finalize Architecture
- Week 2: Verify AES Core
- Week 3: Develop logic-locking FSM that enables AES core
- Week 4: Initial control-path integration and command interface refinement
- Week 5: Key Vault Integration
- Week 6: Logic validation and subsystem debugging
- Week 7: Subsystem Verification

- Week 8: Full-system testbench
- Week 9: Full Integration
- Week 10: Synthesis and timing/area analysis
- Week 11: Security validation
- Week 12: Finalize report/project and presentation

6.) Roles

- Carly will be focusing on the system architecture and control path
- Wills focus will be on datapath integration and verification
- We expect to collaborate across all parts of the project

7.) Summary of Completed Work

AES Core Simulation and Verification:

The AES-128 encryption core has been successfully simulated and functionally verified. Encryption operations were validated against expected test vectors to confirm correctness of the datapath implementation, including round transformations and key addition stages. This establishes a stable cryptographic engine for subsequent security integration.

Module-Level Verification Infrastructure:

Early testbenches have been developed to validate standalone subsystem functionality. These include stimulus generation, signal monitoring, and correctness checking at the module level.

128 Bit Comparator:

Module which takes in two keys, one should be written from an untrusted CPU, the other should be read from an eFuse or other type of secure memory.

128 Bit Comparator Assertions

```
// "Reference Model"
function automatic logic ref_match(input logic [SIZE-1:0] a, input logic [SIZE-1:0] b);
    ref_match = (a == b); // SV will return logic high if a==b
endfunction

// Task to check in a single step
task automatic check(string name);
    logic exp;
    exp = ref_match(key_in, key_hw);
    #0; //short delay
    if (match != exp) begin
        $fatal(1, "[%0t] %s FAILED: key_in=%h key_hw=%h match=%b exp=%0b",
            $time,name,key_in,key_hw,match,exp); // end sim if we hit this cond
    end
    else begin
        $display("[%0t] %s PASS : match=%b", $time, name, match);
    end
endtask
```

128 Bit Comparator Test Sequence

```

// Test
initial begin
    $display("=== begin testbench sequence ===");

    // 1) Equal keys we expect a 1 out
    key_hw = 128'hDEADBEEF_F00DFACE_01234567_89ABCDEF;
    key_in = key_hw;
    check("equal_keys");

    // 2) single bit mismatch
    key_in = key_hw ^ (128'h1 << 7); // shift 1 bit
    check("one_bit_mismatch");

    // 3 All zeroes -> expect match
    key_in = '0;
    key_hw = '0;
    check("all_zeros_equal");

    // 4) Zeros vs Ones mismatch
    key_in = '1;
    key_hw = '0;
    check("zero_vs_one");

    // 5) Random tests
    for (int i = 0; i < 200; i++) begin
        key_hw = {$urandom, $urandom, $urandom, $urandom}; // 128 bits
        // 50/50 chance of match
        if ($urandom_range(0,1)) key_in = key_hw;

        else key_in = {$urandom, $urandom, $urandom, $urandom};

        check($sformatf("random_%0d", i));
    end

    $display("=== ALL TESTS PASSED ===");
    $finish;

```

Initial Key Vault Implementation:

A preliminary ROM-based “Key Vault” module has been designed and simulated. The module currently supports static key storage and controlled readout. This serves as the structural foundation for secure key delivery to the AES engine.

Key Vault Delivery Logic - While the ROM structure is operational, secure key delivery mechanisms are still under development. Planned enhancements include controlled output enable, sequencing with the unlock FSM, and zeroization capability.

ROM Assertions

```

task automatic tb_expect(string name, logic actual, logic expected);
    if (actual != expected) begin
        $display("FAIL: %s at t=%0t actual=%b expected=%b", name, $time, actual, expected);
    $finish;

```

```

end else begin
    $display("PASS: %s at t=%0t", name, $time);
end
endtask

task automatic tb_expect32(string name, logic [31:0] actual, logic [31:0] expected);
    if (actual !== expected) begin
        $display("FAIL: %s at t=%0t actual=0x%08h expected=0x%08h", name, $time, actual,
expected);
        $finish;
    end else begin
        $display("PASS: %s at t=%0t data=0x%08h", name, $time, actual);
    end
endtask

// Pulse a read transaction (one oe rising edge)
task automatic do_read(input logic [1:0] idx, input logic [31:0] exp_data);
    // set up address while disabled
    @(negedge clk);
    word_idx = idx;

    // start transaction: raise enables before a posedge
    @(negedge clk);
    chip_en = 1'b1;
    read_en = 1'b1;

    // sample on next posedge + small delay
    @(posedge clk); #1;

    tb_expect($sformatf("oe asserted for idx=%0d", idx), oe, 1'b1);
    tb_expect32($sformatf("data_out matches idx=%0d", idx), data_out, exp_data);

    // end transaction: deassert read_en to create a new oe rising edge next time
    @(negedge clk);
    read_en = 1'b0;
    chip_en = 1'b0;

    // allow one posedge to settle
    @(posedge clk); #1;
    tb_expect("oe deasserted after transaction", oe, 1'b0);
endtask

// Attempt a read after lockout; expect oe low and data_out zero
task automatic expect_locked_read(input logic [1:0] idx);
    @(negedge clk);
    word_idx = idx;

    @(negedge clk);
    chip_en = 1'b1;
    read_en = 1'b1;

    @(posedge clk); #1;
    tb_expect($sformatf("oe stays low after lockout idx=%0d", idx), oe, 1'b0);
    tb_expect32("data_out stays zero when disabled", data_out, 32'h0000_0000);
endtask

```

```
@(negedge clk);
read_en = 1'b0;
chip_en = 1'b0;
endtask
```

ROM Test Sequence

```
initial begin
    $display("=== Begin ROM TB ===");

    // init
    rst_n = 1'b0;
    chip_en = 1'b0;
    read_en = 1'b0;
    zeroize = 1'b0;
    word_idx= 2'd0;

    $dumpfile("tb_key_vault_rom.vcd");
    $dumpvars(0, tb_key_vault_rom);

    // reset release
    repeat (2) @(negedge clk);
    rst_n = 1'b1;
    @(posedge clk); #1;

    // Sanity: disabled outputs
    tb_expect("oe low when disabled", oe, 1'b0);
    tb_expect32("data_out zero when disabled", data_out, 32'h0000_0000);
    tb_expect("tamper not latched after reset", tamper_latched, 1'b0);
    tb_expect("read_once not latched after reset", read_once_latched, 1'b0);

    // Read all 4 words (big-endian order)
    do_read(2'd0, 32'h00112233);
    do_read(2'd1, 32'h44556677);
    do_read(2'd2, 32'h8899AABB);
    do_read(2'd3, 32'hCCDDEEFF);

    // After 4 reads, read_once should be latched (may latch at/after 4th read posedge)
    @(posedge clk); #1;
    tb_expect("read_once latched after 4 reads", read_once_latched, 1'b1);

    // 5th read should be blocked
    expect_locked_read(2'd0);

    // Reset again, then test zeroize tamper lock
    $display("=== Reset + zeroize test ===");
    @(negedge clk);
    rst_n = 1'b0;
    chip_en = 1'b0;
    read_en = 1'b0;
    zeroize = 1'b0;
    repeat (2) @(negedge clk);
    rst_n = 1'b1;

    // Trip zeroize
```

```

@(negedge clk);
zeroize = 1'b1;
@(posedge clk); #1;
tb_expect("tamper latched after zeroize", tamper_latched, 1'b1);

// Attempts should be blocked
expect_locked_read(2'd2);

$display("=== ALL ROM TB TESTS PASSED ===");
$finish;
end

```

Key Write Buffer Assertions

```

task automatic tb_expect_bit(string name, logic actual, logic expected);
    if (actual !== expected) begin
        $display("FAIL: %s at %t actual=%h expected=%h", name, $time, actual, expected);
        $finish;
    end
    else begin
        $display("PASS: %s at t=%0t", name, $time);
    end
endtask

task automatic tb_expect_vec128(string name, logic [127:0] actual, logic [127:0] expected);
    if (actual !== expected) begin
        $display("FAIL: %s at t=%0t actual=%h expected=%h", name, $time, actual,
expected);
        $finish;
    end else begin
        $display("PASS: %s at t=%0t", name, $time);
    end
endtask

task automatic tb_expect_mask(string name, logic [3:0] actual, logic [3:0] expected);
    if (actual !== expected) begin
        $display("FAIL: %s at t=%0t actual=%h expected=%h", name, $time, actual,
expected);
        $finish;
    end else begin
        $display("PASS: %s at t=%0t", name, $time);
    end
endtask

// bus helpers (place holders for real signals)
task automatic do_reset();
    begin
        @(negedge clk);
        reset = 1'b1;
        zeroize = 1'b0;
        wr_en = 1'b0;
        commit = 1'b0;
        @(negedge clk);
        @(negedge clk);
    end
endtask

```

```

        reset = 1'b0;
        @(negedge clk);
    end
endtask

task automatic write_word(input logic [1:0] addr, input logic [31:0] data);
    begin
        @(negedge clk)
        wr_addr = addr;
        wr_data = data;
        wr_en = 1'b1;
        commit = 1'b0;
        @(negedge clk);
        wr_en = 1'b0;
    end
endtask

task automatic do_commit();
    begin
        @(negedge clk);
        commit = 1'b1;
        wr_en = 1'b0;
        @(negedge clk);
        commit = 1'b0;
    end
endtask

// handler if commit and write on same cycle ( commit should win)
task automatic commit_conflicts_write(input logic [1:0] addr, input logic [31:0] data);
    begin
        @(negedge clk);
        wr_addr = addr;
        wr_data = data;
        wr_en = 1'b1;
        commit = 1'b1;
        @(negedge clk);
        wr_en = 1'b0;
        commit = 1'b0;
    end
endtask

task automatic sample_idle_outputs(string tag);
    begin
        @(posedge clk); #1;
        tb_expect_bit({tag, ": do compare low"}, do_compare, 1'b0);
        tb_expect_bit({tag, ": commit_illegal low"}, commit_illegal, 1'b0);
    end
endtask

```

WIP Key Buffer Test Sequence

```

// test sequence
initial begin
    $display(" === begin test squencece ===");
    // initial inputs

```

```

reset = 1'b0;
zeroize = 1'b0;
wr_en = 1'b0;
wr_addr = 2'd0;
wr_data = 32'h0;
commit = 1'b0;

// built in hw key -> big endian
key_hw = 128'h00112233_44556677_8899AABB_CCDDEEFF;
$dumpfile("tb_key_commit_buffer.vdc");
$dumpvars(0, tb_key_commit_buffer);

// 1.) reset
$display("Test 1: Reset");
do_reset(); // use do reset task
tb_expect_mask("reset: valid_mask=0000", valid_mask, 4'b0000);
tb_expect_bit("reset: all_valid=0", all_valid, 1'b0);
tb_expect_bit("reset: do_compare=0", do_compare, 1'b0);
tb_expect_bit("reset match_q=0", match_q, 1'b0);
tb_expect_bit("reset: commit illegal=0", commit_illegal, 1'b0);
tb_expect_vec128("reset: key_in=0", key_in, 128'h0);
$display("Test 1: PASS");
$finish;
end

```

FSM based Arbitration

The arbiter is being extended to operate in conjunction with the logic-locking FSM, ensuring encryption resources remain inaccessible prior to successful authentication.

Arbiter/Control Module Design - Development has begun on the system arbiter responsible for mediating access between the command interface and encryption datapath. A basic arbitration module has been simulated; however, integration of true logic-locking enforcement remains in progress.

FSM Arbiter Assertions

```

// MUTUAL EXCLUSION: Never grant both
a_mutex : assert property (@(posedge clock) disable iff (cb.reset ||
$isunknown({cb.gnt_0,cb.gnt_1})))
!(cb.gnt_0 && cb.gnt_1)
) else $fatal(1, "Arbiter Failure: Both grants high at t=%0t", $time);

// False high check : if gnt_0 then we expect req_0
false_high_gnt0 : assert property (@(posedge clock) disable iff (cb.reset ||
$isunknown({cb.gnt_0,cb.req_0})))
cb.gnt_0 |-> $past(cb.req_0)
) else $fatal(1, "gnt_0 high while req_0 low at t=%0t", $time);

// if gnt_1 we expect req_1
false_high_gnt1 : assert property (@(posedge clock) disable iff (cb.reset ||
$isunknown({cb.gnt_1,cb.req_1})))
cb.gnt_1 |-> $past(cb.req_1)
) else $fatal(1, "gnt_1 high while req_1 low at t=%0t", $time);

a_no_tail0 : assert property (@(posedge clock)

```

```

        disable iff (reset || $isunknown({gnt_0, req_0}))
        !(gnt_0 && req_0)
    ) else $fatal(1, "Tail detected: gnt_0=1 while req_0=0 at t=%0t", $time);

a_no_tail1 : assert property (@(posedge clock)
    disable iff (reset || $isunknown({gnt_1, req_1}))
    !(gnt_1 && req_1)
) else $fatal(1, "Tail detected: gnt_1=1 while req_1=0 at t=%0t", $time);

```

Test Sequence for Arbiter

```

// test stimuli
initial begin
    $display(" === Begin Test Seq == ");
    // init signals
    reset = 0;
    req_0 = 0;
    req_1 = 0;

    // dump waveforms
    $dumpfile("fsm_one.vcd");
    $dumpvars(0, tb_fsm_using_one_always);

    // Hold reset for a few cycle
    @(negedge clock);
    reset = 1;
    @(negedge clock);
    @(negedge clock);
    reset = 0;
    @(negedge clock);

    // request 0 for 2 cycles
    req_0 = 1;
    @(posedge clock); #1;
    tb_expect("no immediate grant", gnt_0, 1'b0);
    tb_expect("gnt_1 stays low when req_0 high", gnt_1, 1'b0);

    // hold for one cycle
    @(posedge clock); #1;
    tb_expect("gnt_0 high after second cycle", gnt_0, 1'b1);
    @(negedge clock);
    req_0 = 1'b0;

    // after following pos edge gnt_0 should fall
    @(posedge clock); #1;
    tb_expect("gnt_0 deasserted immediately", gnt_0, 1'b0);

    @(posedge clock); #1;
    tb_expect("gnt_0 deserted one cycle after req_0 low", gnt_0, 1'b0);

    @(negedge clock);
    @(negedge clock);

```

```

    @(negedge clock);

    // test req/gnt 1
    reset = 1;
    @(negedge clock);
    @(negedge clock);
    reset = 0;
    @(negedge clock);
    // request 1
    req_1 = 1;
    @(posedge clock); #1;
    tb_expect("No immediate grant",gnt_1, 1'b0);
    tb_expect("gnt_0 stays low when req_1 high",gnt_0, 1'b0);
    // hold for one cycle
    @(posedge clock); #1;
    tb_expect("gnt_1 high after one cycle", gnt_1, 1'b1);
    @(negedge clock);
    req_1 = 1'b0;
    @(posedge clock); #1;
    // after following positive edge we expect gnt_1 to fall
    tb_expect("gnt_1 low after one cycle", gnt_1, 1'b0);

    $display("=== Finish Test Seq === ");
    $finish;

end

// monitor signals
initial begin
    $display("time  clk rst req0 req1 | gnt0 gnt1 | state");
    $monitor("%4t   %0b %0b %0b %0b | %0b %0b | %3b",$time, clock, reset, req_0,
req_1, gnt_0, gnt_1, dut.state);
end

```

Key Generation: In Progress

Implementing the On-chip key generation using a TRNG/PRNG seeded inside the chip

- generate a 128-bit key internally using a PRNG (LFSR/CTR-DRBG style) seeded from some entropy source.
- a. PRNG core (fast, synthesizable)
 - Implement one of these:
 - 128-bit LFSR (simplest)
 - Xorshift (also simple)
 - AES-CTR DRBG (most “correct” but heavier)
 - LFSR as PRNG + “seed” from something noisy-ish
- b. Entropy / seed source (the hard part)
 - In FPGA/RTL simulation, “true” entropy is tricky. Common class-acceptable approaches:
 - Seed sources (choose 1–2):
 - Ring oscillator jitter TRNG (realistic but FPGA/ASIC dependent)
 - SRAM startup state (ASIC realistic; FPGA depends)

- Free-running counter sampled on asynchronous event (button press / host timing variability)
- Multiple oscillators XOR'd (TRNG-ish)
- If no real entropy available: accept host-provided seed, but generate the key internally

8.) Bibliography

[1] D. Evans, K. Brown, and P. Bond, "FIPS 197 Federal Information Processing Standards Publication Advanced Encryption Standard (AES)," *Advanced Encryption Standard (AES)*, Nov. 2001, doi: <https://doi.org/10.6028/NIST.FIPS.197-upd1>.

[2] "TPM 2.0 Library," *Trusted Computing Group*.
<https://trustedcomputinggroup.org/resource/tpm-library-specification/>

[3] "TCG Trusted Platform Module Library Part 1: Architecture Family '2.0' TCG Published." Available:
<https://trustedcomputinggroup.org/wp-content/uploads/TPM-2.0-1.83-Part-1-Architecture.pdf>

[4] J. Rajendran, Y. Pino, Ozgur Sinanoglu, and R. Karri, "Security analysis of logic obfuscation," *Proceedings of the 49th Annual Design Automation Conference*, Jun. 2012, doi: <https://doi.org/10.1145/2228360.2228377>.

[5] Swarup Bhunia *et al.*, "Hardware IP Protection against Confidentiality Attacks and Evolving Role of CAD Tool," *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*, pp. 1–8, Oct. 2022, doi: <https://doi.org/10.1145/3508352.3561103>